

LC 518: Coin Change - II

Amdocs

Park+

HSBC

ZS Associates

Morgan Stanley

Problem Understanding

We are given an array of coins and an amount. The goal is to return the total number of ways to make up that amount using the given coins. Since we need to calculate the number of ways to reach a specific state, it is naturally a **Dynamic Programming (DP)** problem.

Defining the DP State

A common pitfall is to assume that the `amount` alone is enough to define the state. Let's look at an example to see why that fails:

Let `amount = 3`

Case 1: `coins = [1, 2, 5]`

Case 2: `coins = [2, 5]`

*These will yield different answers, proving that the amount alone is **not enough**.*

Therefore, the DP state must track both the current coin we are considering and the remaining amount:

DP State → (current coin index, remaining amount)

Choices and Recurrence Relation

At every coin index (let's call the index `idx`), we have two choices:

- **(A) Take the current coin:** Because we have an infinite supply of each coin, we can pick the coin and stay on the same index. State becomes:

```
solve(idx, amount - coins[idx])
```

- **(B) Skip the current coin:** Move to the next coin without changing the amount. State becomes:

```
solve(idx + 1, amount)
```

The total ways for a given state is the sum of these two choices:

```
dp[idx][amount] = solve(idx, amount - coins[idx]) +  
                  solve(idx + 1, amount)
```

Base Cases

- `amount == 0` → Return `1` (We have successfully found a valid combination).
- `idx == n` → Return `0` (We have run out of coins without making up the amount).

C++ Implementation

Here is the implementation of the above logic using Top-Down Dynamic Programming (Memoization). Note that the code retains its original logic and comments.

```

class Solution {
public:
    vector<vector<int>> dp;
    int n;

    int solve(int idx, int amount, vector<int>& coins) {
        if (amount == 0) {
            return 1;
        }

        if (idx == n) {
            return 0;
        }

        if (dp[idx][amount] != -1) {
            return dp[idx][amount];
        }

        int ways = 0;
        if (amount >= coins[idx]) {
            ways += solve(idx, amount - coins[idx], coins); // ya to coin lek
        }

        ways += solve(idx + 1, amount, coins); // ya to coin skip krke dekh lo

        dp[idx][amount] = ways;
        return ways;
    }

    int change(int amount, vector<int>& coins) {
        n = coins.size();
        dp.assign(n + 1, vector<int>(amount + 1, -1));

        return solve(0, amount, coins);
    }
};

```